An Endogenous Modal-Depth Architecture for Safety Enforcement in Autonomous Decision Systems

Anish Chilakala

Abstract

As Autonomous Decision Systems become increasingly complex, ensuring reliable safety enforcement within their software logic and cognitive reasoning processes becomes increasingly important. This paper investigates a new theoretical endogenous method for enforcing safety in Autonomous Decision Systems through cognitive reasoning and software logic. The method is positioned within the action pipeline, where candidate actions are selected and authorized, rather than as an external monitor that reacts after execution. To preserve the feasibility focus of the study, the architecture deliberately excludes sensor and perception data; binding the evaluation to a particular sensing stack would shift the research question from whether the idea is feasible to whether it works for one specific system. The proposed method was evaluated within a generalized Haskell sandbox environment designed to replicate the specific part of the ADS required for experimental evaluation. The sandbox uses stochastic software variability to simulate uncertainty representative of operational ADS environments. Under ground-truth evaluation, the proposed architecture achieved approximately 95% safety-enforcement accuracy across 1,000,000 experimental runs, with the remaining 5% discrepancy identified as epistemic uncertainty. Under the observed_context evaluation, which evaluates the architecture using only the information available to the system rather than inaccessible ground-truth information, the experimental framework achieved 100% decision accuracy relative to its available observations, demonstrating that the architecture behaved consistently with the information it could access. This work contributes a feasible endogenous approach to safety enforcement in Autonomous Decision Systems and provides a generalized experimental framework for extended future investigation and evaluation of this safety architecture.

1 Introduction

1.1 Background

Autonomous Decision Systems (ADSs) are becoming increasingly capable of performing complex tasks with limited or no human intervention. As these systems grow in capability and complexity, the need for reliable safety mechanisms becomes increasingly important [1], [2], [3]. This is especially relevant as autonomous reasoning and decision-making pipelines become more powerful and more difficult to constrain through conventional approaches [2].

1.2 Motivation

Recent developments in Autonomous Decision Systems have further emphasized the importance of investigating robust safety architectures. One motivating example is Claude Mythos Preview, a model documented in Anthropic's own reporting [8]. According to that documentation, after an initial user prompt Mythos Preview could autonomously discover and exploit zero-day vulnerabilities in major operating systems and web browsers, a dual-use capability that led Anthropic to restrict

the model's release to a small number of partners for defensive use [8]. Anthropic's alignment risk assessment for the same model further observes that it can sometimes employ concerning actions to work around obstacles to task success, that it is used more autonomously and agentically than any prior model, and that its strength in software engineering and cybersecurity makes it more capable at working around restrictions [9].

These behaviors were observed in controlled evaluation settings rather than in deployed operation, and they are cited here as motivation rather than as evidence that the proposed architecture would have prevented them. They are relevant because they show concretely how an increasingly capable autonomous system can pursue actions outside its intended operational constraints, which is precisely the class of problem this paper seeks to address. For that reason, they motivated the investigation presented here into whether alternative safety architectures can provide the support needed to control autonomous actions and abilities.

1.3 Why Software Logic and Cognitive Reasoning?

The motivation described above explains why this paper focuses on software logic and cognitive reasoning rather than external hardware or sensor behavior. The risk being considered is not only that an autonomous system receives imperfect information, but that the system's own decision process may allow an action outside its intended constraints. As the documentation referenced in §1.2 indicates, a sufficiently capable system may take concerning actions to work around obstacles to task success, including actions its operators did not intend it to perform [8], [9]. Failures of this kind originate inside the decision-making process, at the point where candidate actions are selected, evaluated, and committed to execution. This stage is referred to here as the action pipeline, and the action pipeline is itself a component of the system's software logic and cognitive reasoning, the layer responsible for interpreting context, evaluating options, and deciding what the system will do.

This observation determines where the proposed method is placed. If the risk is that an autonomous system commits to an action it should not perform, then a safety mechanism intended to prevent that outcome must operate at or before the action-selection stage, inside the same reasoning pipeline that produces the action, rather than as a separate observer that reacts only after execution has begun. The proposed modal-depth architecture is therefore positioned as an endogenous component of this pipeline: each candidate action is evaluated through the modal structure before it is permitted to proceed to execution. By embedding enforcement within the reasoning process, the architecture treats safety as a property of the decision itself rather than a check applied after the fact.

It is also important to distinguish the target of this work from perception. The architecture governs whether a candidate action is permitted to proceed given the system's internal representation of its situation; it does not interpret raw sensor measurements, fuse perception signals, or estimate physical state directly. The input to the enforcement layer is the system's existing internal representation, the same information the action pipeline already uses, rather than a new sensing modality. This separation is deliberate, and the reasons for excluding sensor data are discussed in §1.6.

1.4 Existing Safety Approaches

Many existing safety implementations operate as external monitoring or validation mechanisms that evaluate system behavior before or after actions are generated [5], [6], [7]. These approaches remain valuable and continue to play an important role in autonomous system safety. This work, however, investigates a different architectural perspective in which safety enforcement is incorporated into

the reasoning process itself. In this setting, safety is intended to act as an endogenous component of the system's decision-making pipeline, working alongside or inside of the decision-making pipeline, influencing whether actions are permitted to proceed, in contrast to approaches that place the safety monitor outside the controlled process [5], [7].

1.5 Proposed Framework

This paper proposes a framework in which modal depth is investigated as one possible endogenous safety axis for Autonomous Decision Systems. In this framework, independent reasoning modalities represent distinct aspects of the system's internal representation. By using thresholds to estimate danger levels and passing this information to a policy layer, the architecture aims to determine when safety enforcement should be applied. The framework is intentionally designed to be general so that it can be adapted to different ADSs through system-specific modality definitions, threshold structures, policy logic, and enforcement behavior, while preserving the core architectural principles. The work is intended to provide a flexible foundation for investigating endogenous safety enforcement rather than a complete or domain-specific solution.

1.6 Scope and Objectives

This paper does not claim to provide a complete solution to autonomous safety, nor does it suggest that modal-depth-based reasoning is inherently superior to existing approaches. Instead, it presents an exploratory study of one possible method for embedding safety directly into an ADS's software logic and cognitive reasoning process within a controlled sandbox environment. The objective is to assess whether such an architecture can meaningfully govern safety-enforcement decisions under conditions that reflect real-world uncertainty while remaining computationally practical. The broader goal is to establish a foundation for future research into whether this approach can be extended and evaluated more fully into real ADS settings.

A second scope decision concerns the deliberate exclusion of sensor and perception data. The present study does not ingest raw or processed sensor streams as inputs to the enforcement architecture. This choice is methodological rather than technical. The central research question of this work is whether an endogenous modal-depth architecture can feasibly enforce safety within the decision pipeline of an Autonomous Decision System. Introducing sensor-specific inputs, such as a particular platform's sensor streams, perception embeddings, or domain-specific state estimators, would bind the evaluation to the noise profile, representation, and failure modes of one specific ADS. Such a binding would shift the research question from whether the architectural idea is feasible to whether the idea works for one specific ADS, a question whose answer would not generalize beyond the system tested and would therefore invalidate the generalized, architecture-level conclusions this paper seeks to establish.

For this reason, the experimental framework represents the system's internal perception only abstractly, through a single noisy `observed_context` derived from a hidden ground-truth value via uniform stochastic variability. This abstraction represents whatever internal representation of the ground-truth (danger level) the system sees. (§4.6).

1.7 Experimental Overview

To evaluate this architecture, a customizable and malleable sandbox environment was developed to simulate the decision-making process of an Autonomous Decision System under stochastic software variability and epistemic uncertainty. The framework was implemented in Haskell and evaluated using multiple modal configurations within a controlled experimental setting. The study aims to

introduce the proposed sandbox framework, investigate modal depth as an endogenous safety axis, evaluate the architecture under conditions representative of real-world constraints, and measure the accuracy of its safety-enforcement decisions through empirical experimentation. These results are intended to inform future work on the scalability and applicability of the approach in more complex and realistic ADS environments.

2 Methodology

2.1 Definitions of Related Concepts

Modal Depth: A safety-enforcement architecture that uses multiple modalities and evaluation layers ("depths").
Modality: A modality represents an independent reasoning dimension, such as safety (d_s), ethics (d_e), action (d_a), or contextual reasoning (d_c). Each modality represents a distinct structural component of the overall system and responds uniquely based on the system's interpretation of a given situation. The number and definitions of active modalities are determined by the specific Autonomous Decision System (ADS) being studied, making the method fully customizable. Modality definitions in the sandbox safety method are completely different and do not mimic how real modalities will be defined. When `observed_context` is shown generating modalities, it is showing how modalities are modeled and defined in order to conduct experimentation in the sandbox. The distinction is explained in subsection 2.3.
Threshold: A threshold is a baseline used to classify an individual modality's danger level relative to its current value. Modality values alone do not inherently determine whether a situation represents high or low danger; the threshold serves as the mathematical separator between these states. In this implementation, a modality value above its threshold represents high danger, while a modality value below its threshold represents low danger. This relationship can be inverted if required by the system being modeled. Threshold values are determined through hyper-parameter tuning and tuning thresholds maximizes safety-enforcement accuracy.
Epistemic Uncertainty: Uncertainty due to a system's lack of knowledge, or the gap between the information known to the system and the ground truth. This is not to be confused with aleatoric uncertainty [4]. Epistemic uncertainty refers to the difference between the system's observation of reality and the actual ground-truth reality. In this paper, it represents the mismatch between the system's internal observation and the external environment.
Aleatoric Uncertainty: Stochastic noise or variability in the system or data itself. In this case, aleatoric uncertainty is a normal and irreplaceable part of software. It affects the system's observation of reality and is modeled as internal noise in the observed context.
Logical Jitter: A subsidiary of aleatoric uncertainty, classified by software fluctuations and decision instability [4].
Multi-Dimensional: Refers to multiple modalities; for example, one dimension corresponds to one modality in the system. In the same way, a 5th dimension would mean having 5 modalities in a system.
Enforcing Safety: A way to make sure the ADS does the action it's supposed to do, but not the action it's not supposed to do.
Deterministic Noise: Randomness in a system that is predictable and fixed noise generated by a mathematical formula. This is what Haskell's randomRIO runs on and is normally paired or interpreted as a uniform distribution.
Stochastic Noise: Random, unpredictable fluctuations in a system or signal. This is normally paired or interpreted as a gaussian distribution. Haskell randomRIO can be used to model this.

2.2 Experiment Structure

To evaluate the framework, a sandbox environment was created to mimic a specific portion of an ADS environment while maintaining experimental control. The objective was to measure how the proposed modal framework responded to uncertainty and software variability [4].

The system introduces stochastic variation through a multiplicative noise factor ranging from 0.85 to 1.15, representing ±15% software variability. This variability is incorporated directly into the observed context definition.

The experiment uses `randomRIO` in Haskell to generate uniformly distributed random values across predefined ranges. The real-world danger context (c) is generated uniformly within the range 0.0 to 10.0 (generated deterministically) and is then transformed into an observed context value through randomRIO modeling to mimic stochastic noise variability of the system ±15%.

```
generateScenario :: IO Scenario
generateScenario = do
    c <- randomRIO (0.0, 10.0)
    stochastic_system_variability <- randomRIO (0.85, 1.15)

    let observed_context = clamp $
            (max 0.0 (min 10.0 (c * stochastic_system_variability)) / 10)

        clamp x = max 0.0 (min 1.0 x)

        logisticMidpoint =
            1.0 / (1.0 + exp (-12.0 * (observed_context - 0.55)))

        d_s = clamp $ (1.0 - logisticMidpoint) + 0.05

        d_a = clamp $ 1.0 - exp (-1.5 * observed_context)

        d_e =
            if d_a > 0.55 || d_s < 0.55
            then clamp observed_context
            else clamp 0.05

        d_c = clamp $ (d_a + d_e + (1.0 - d_s)) / 3.0

    return (Scenario c d_a d_e d_c d_s observed_context)
```

Four modalities are used throughout the experiment: d_s (safety modality), d_a (action modality), d_e (ethical modality), and d_c (contextual modality). The contextual modality sums all the information from the remaining modalities to determine how the system as a whole is responding to the observed context.

The framework defines several data types that are used throughout the experiment.

```
data Scenario = Scenario
  { c                :: {-# UNPACK #-} !Double
  , d_a              :: {-# UNPACK #-} !Double
  , d_e              :: {-# UNPACK #-} !Double
  , d_c              :: {-# UNPACK #-} !Double
  , d_s              :: {-# UNPACK #-} !Double
  , observed_context :: {-# UNPACK #-} !Double
  } deriving (Show, Eq)
```

Scenario stores the real-world context, the observed context variable, and modality definitions

to be used across the whole entire sandbox system.

```
data ModeSet = ModeSet
  { d_a_active :: !Bool
  , d_e_active :: !Bool
  , d_c_active :: !Bool
  , d_s_active :: !Bool
  } deriving (Show, Eq)
```

ModeSet determines which modalities are active during a given simulation and enables the ablation studies used to evaluate each unimodal accuracy to, therefore, help single out problems in the code or logic, and to tune threshold values.

```
data Action = ACT | NO_ACT
  deriving (Show, Eq)
```

Action defines the two possible safety-enforcement decisions available to the policy layer. This is standard to a generic ADS. What ACTing to enforce safety does and how the system ACTs to enforce safety are purely based on the system in question.

```
data Result = Result
  { action  :: !Action
  , correct :: !Bool
  } deriving (Show, Eq)
```

Result stores the policy decision and whether that decision was evaluated as correct by the evaluator. Strictness annotations (`!`) and `UNPACK` pragmas are used to reduce heap allocation and improve runtime performance during large-scale simulation runs and in real time performance.

The simulation is executed through the experimental driver:

```
runSimulation :: ModeSet -> Int -> IO [Result]
runSimulation modeset n =
    fmap (map (processScenario modeset))
         (Scenario.generateScenarios n)

generateScenarios :: Int -> IO [Scenario]
generateScenarios _ = return []
```

The `generateScenarios` stub shown above is a defensive fallback included to terminate the run gracefully and prevent indefinite loading in the event of a generation error; the full generator is used in the executed configuration.

2.3 Experimental System Overview

The framework operates using the `observed_context` rather than direct access to the underlying real-world context (c). During early experimentation, evaluation procedures were initially based on the real-world context. During this early experimentation process, large fluctuations in accuracy occurred.

To reduce the effects of these fluctuations (which were later identified as epistemic uncertainty), the framework evolved from a single-threshold architecture into a multi-threshold system. The resulting tri-threshold architecture applies different threshold regions based on the value of the observed context, reducing classification uncertainty and instability near decision boundaries.

The framework follows the sequence shown in Figure 1:

1. Generate a ground-truth context c (used only to derive `observed_context` and for external evaluation).
2. Derive `observed_context` from c via stochastic variability.
3. Compute modality values (d_s, d_a, d_e, d_c) from `observed_context`.
4. Select thresholds based on the `observed_context` regime (tri-threshold and is based on the evaluator regimes; ex. the different regimes in the evaluator in the sandbox).
5. Classify each modality as high- or low-danger relative to its threshold value.
6. Apply policy logic to the determined danger classifications.
7. Emit `ACT` (enforce safety intervention) or `NO_ACT` (action pipeline proceeds unmodified).

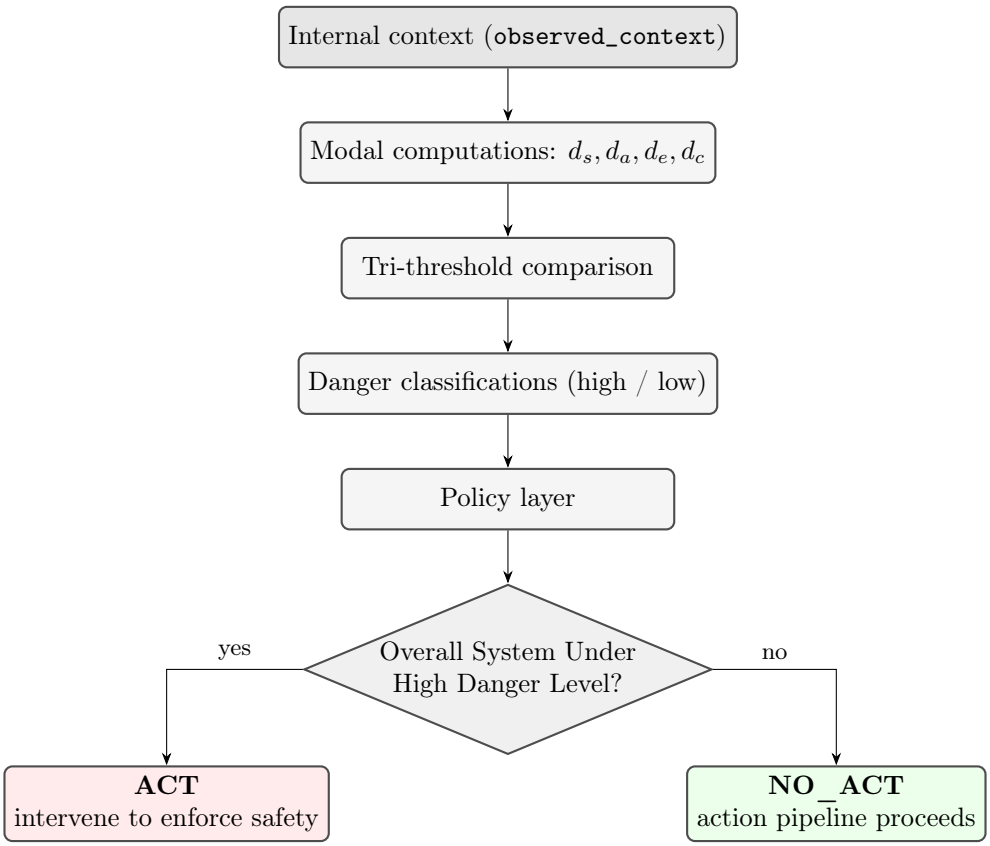


Figure 1: Endogenous enforcement pipeline. Safety is evaluated within the reasoning process before an action proceeds to execution.

Although modality definitions may vary between ADS implementations, modality values alone do not determine whether a situation represents high danger or low danger. Thresholds perform this classification step. In the current implementation, a modality value above its threshold represents high danger, while a modality value below its threshold represents low danger.

```
thresholds :: Scenario -> (Double, Double, Double, Double)
thresholds scenario
  | observed_context scenario >= 0.55 =
```

```
    (0.0, 0.0, 0.0, 0.0)

| observed_context scenario >= 0.10 &&
  observed_context scenario < 0.55 =
    (1.0, 1.0, 1.0, 0.0)

| otherwise =
    (1.0, 1.0, 1.0, 0.0)
```

Despite using multiple threshold regions, the underlying principle remains unchanged: thresholds act as context-dependent baselines that calculate and separate high-danger and low-danger states from each other. Figure 2 visualizes the three `observed_context` regimes and the threshold tuple applied in each. Different tuples can be assigned to different threshold regions to adapt to a specific ADS's modalities.

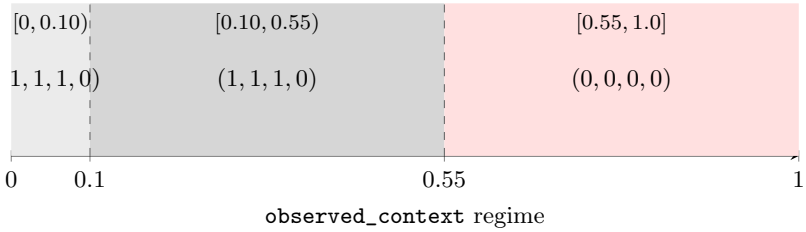


Figure 2: Tri-threshold regions over `observed_context`. The two lower regimes share a threshold tuple; the upper regime applies the zero-valued threshold tuple.

The framework is intentionally designed to be highly customizable. Researchers may modify modality definitions, threshold structures, policy rules, evaluator logic, and the number of modalities while preserving the overall architecture.

Rather than enforcing safety directly from a single contextual value, the framework routes context through multiple reasoning dimensions, converts the resulting outputs into danger classifications using thresholds, and then applies policy logic to determine whether safety enforcement should occur. The dependency structure among the modalities is shown in Figure 3.

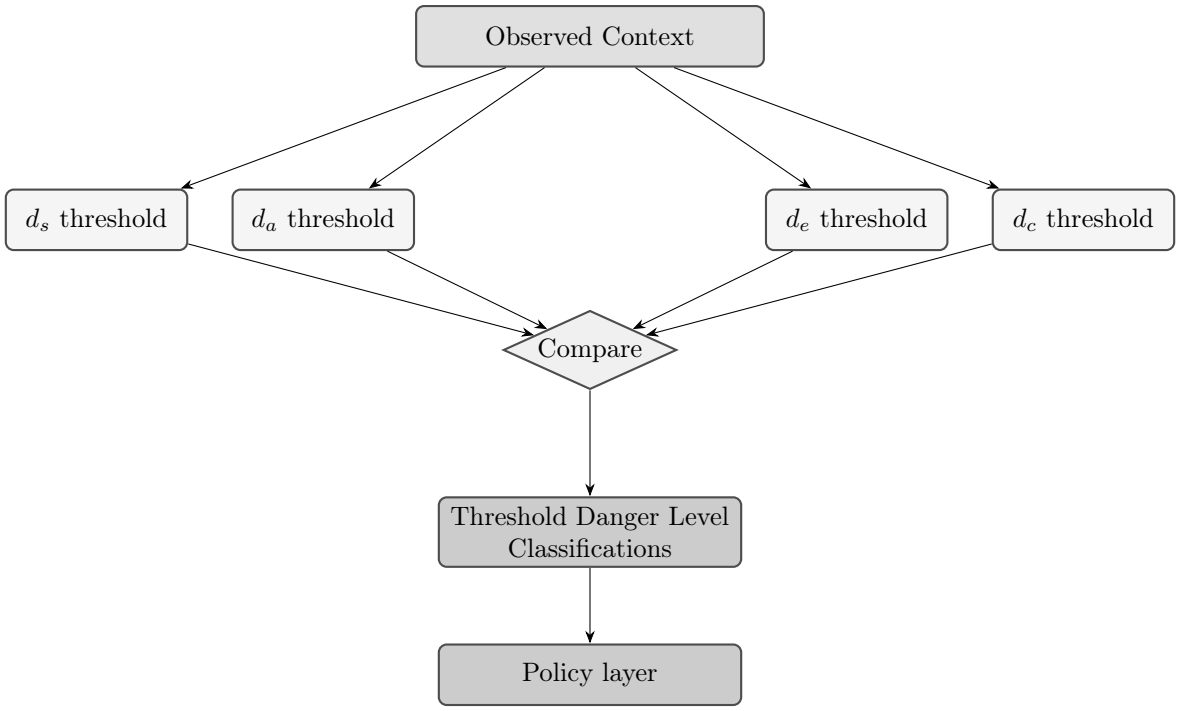


Figure 3: Modality values d_s, d_a, d_e, and d_c are compared against their respective thresholds to determine whether each modality is classified as high danger or low danger. The resulting danger classifications are then passed to the policy layer.

In this sandbox, `observed_context` serves as a simplified internal representation of danger that allows modalities to be defined and evaluated. It is a modeling device specific to the experimental environment rather than a universal requirement of the architecture. In real autonomous decision systems, modalities would not be defined in this way; instead, specific components of the action pipeline (such as planner outputs, risk scores, and task-specific constraints) would themselves serve directly as modalities. In such systems, a variable or signal in a real system that plays a similar role to `observed_context` might exist as one of these modalities, but it would not be used to define other modalities as in the sandbox. The sandbox safety method uses `observed_context` only because the environment does not provide a real action pipeline to draw from and it is used to determine when safety enforcement is needed and when it is not. It is important to maintain the distinction that `observed_context` is not a universal variable and does not represent how modalities are defined in real systems; it is simply the mechanism used to define modalities in this sandbox system because of the absence of a real action pipeline.

When the raw data coming from the modalities is in a format that cannot be interpreted by the threshold, than that data will be converted into a language that can be understood and therefore interpreted by the threshold so the threshold can accurately enforce safety.

When the `observed_context` is gone, the way we also define the tri threshold system also changes. The system would use a tri-threshold system using whatever modality represents safety risk score etc. This safety risk score would be either a modality that represents a safety risk score like the `observed_context` or a collection of modalities to create a singular safety risk score.

The use of `observed_context`, or a direct representation of it, is not required by the tri-threshold approach. It is used in this sandbox implementation because it provides a single value that both corresponds to the system's decision to ACT or NO ACT.

When implemented in a real ADS, this architecture might be complicated to tune due to var-

ious factors such as defining an `observed_context` representation, figuring out proper values and variables that are able to represent and correspond to the system's acting decision etc., and thus might work less efficiently, so a more efficient implementation is presented in Subsection 4.6. Future Work.

The policy layer receives high-danger and low-danger classifications from each modality. These classifications are weighted according to modal importance, allowing future implementations to emphasize the importance of specific modalities when computing final danger scores.

The final policy decision is generated by comparing the aggregated danger information and selecting either:

`ACT:` enforce safety intervention.

`NO_ACT:` no safety intervention required; actions being executed by the action pipeline are not changed or modified in any way.

The framework intentionally leaves `ACT` unspecified: what enforcement does and how it is performed are system-dependent and must be defined per ADS. Leaving `ACT` abstract both isolates evaluation of the core decision logic and prevents the architecture from being biased toward a particular class of ADSs.

This architecture creates a flexible and noise-tolerant ADS safety testing environment capable of evaluating safety-enforcement behavior under conditions of epistemic uncertainty, aleatory uncertainty and stochastic software noise variation.

3 Results

3.1 Accuracy Under Ground-Truth Evaluation

The final experimental configuration produced approximately 95% accuracy when evaluated against real-world ground-truth context (c) using the standard evaluator. During testing, a persistent performance gap of approximately 5% remained despite repeated modifications to modality definitions, threshold tuning, stochastic-noise filtering, policy optimization, and runtime optimization procedures. Subsequent analysis identified the source of this discrepancy as epistemic uncertainty arising from differences between the system's internal perception of the environment using `observed_context` and the underlying ground-truth context (c).

3.2 Ground Truth Evaluation and Logical Jitter

A key result of the study was the persistent discrepancy between 100% accuracy under the observed-context evaluator and approximately 95% accuracy under the ground-truth evaluator. Otherwise known as a persistent 5% bottleneck. Throughout development, numerous potential causes were investigated, including threshold selection, stochastic noise handling, modality formulation, runtime optimization, and policy design. Despite repeated modifications to these components, the discrepancy remained largely unchanged.

When evaluated against the ground truth, the architecture encountered a strict 5% accuracy bottleneck. Even after extensive testing with noise filters to lower the bottleneck, the accuracy bottleneck remained. The $observed_context$ values continued to vary near critical decision boundaries, and the evaluator and runtime system could still disagree about whether a scenario should be classified as dangerous.

For example, in a scenario where the ground-truth value is 5.4 and the high-danger threshold is defined as $5.5 \geq 5.5$, stochastic noise can shift the $observed_context$ to 5.6. Because of this, the

system executes an action to enforce safety based on its perceived threat, even though the true underlying state was not under what is considered to be high danger levels.

Figure 4 illustrates this effect: near the 0.55 high-danger threshold, the ±15% variability band is on the boundary, and the shaded region marks where the evaluator and the runtime system can disagree.

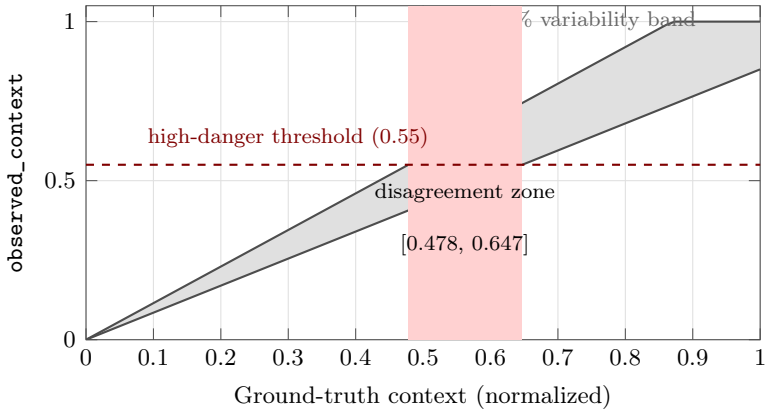


Figure 4: Logical jitter near the decision boundary. The shaded band is the ±15% uniform variability range at each specific ground-truth value; the red region is where the disagreement between the `observed_context` and the ground truth causes the `observed_context` values to fluctuate across the high danger threshold of 0.55 in the graph (the source of the 5% bottleneck).

In the constructed experiment, there are three distinct evaluation points:

```
processScenario :: ModeSet -> Scenario -> Result
processScenario modeset scenario =
    let action = Policy.policy scenario
        correct = (action == ACT && (c scenario >= 0.55 && c scenario <= 10)) || (action ==
    NO_ACT && (c scenario < 1.0)) || (action == NO_ACT && (c scenario >= 1.0 && c scenario < 5.5)
    )
    in Result action correct
```

These small differences between ground truth and `observed_context` occur at each evaluation point.

3.3 Observed-Context Evaluation Results

After extensive testing, the final implementation adopted an evaluator based entirely on the system's observed context. In the implementation, this value is represented by `observed_context` and is used to determine threshold values at each regime. Under this evaluation paradigm, the core decision architecture achieved 100% accuracy relative to the information available to the system and produced zero decision-logic failures. The architecture evaluator confirmed that the modal-depth framework consistently executed the correct deterministic action relative to the real-time information available to the system at runtime.

These outcomes establish a clear distinction between internal runtime execution evaluation and external validation evaluation. When measured against the internal `observed_context`, the evaluator achieved 100% accuracy relative to its available observations; when validated against external ground-truth context (c), it achieved 95% accuracy. This divergence quantifies the exact boundaries of environmental epistemic uncertainty within the system. Figure 5 summarizes the two evaluation paradigms and the gap between them.

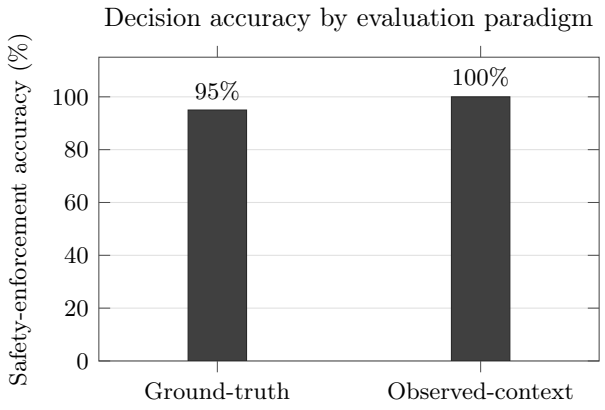


Figure 5: Accuracy under the two evaluation paradigms. The 5% gap corresponds to epistemic uncertainty, not architectural failure.

3.4 Runtime Performance Evaluation

Performance testing was conducted on the compiled Haskell implementation using optimization level (`-O2`) with GHC runtime statistics tracking enabled. The executable was benchmarked using the following compilation and execution parameters:

```
ghc -O2 -rtsopts -fforce-recomp Main.hs -o sim_test
./sim_test +RTS -A600m -s
```

```
 538,621,976 bytes allocated in the heap
       3,272 bytes copied during GC
      44,328 bytes maximum residency (1 sample(s))
      25,304 bytes maximum slop
         627 MiB total memory in use (0 MiB lost due to fragmentation)

                                     Tot time (elapsed)  Avg pause  Max pause
Gen  0         0 colls,     0 par    0.000s   0.000s     0.0000s    0.0000s
Gen  1         1 colls,     0 par    0.000s   0.001s     0.0010s    0.0010s

INIT    time    0.067s  (  0.085s elapsed)
MUT     time    0.282s  (  0.282s elapsed)
GC      time    0.000s  (  0.001s elapsed)
EXIT    time    0.001s  (  0.001s elapsed)
Total   time    0.351s  (  0.369s elapsed)

%GC     time       0.0%  (0.0% elapsed)
Alloc rate    1,907,186,805 bytes per MUT second
Productivity  80.5% of total user, 76.6% of total elapsed
```

The runtime profile indicates exceptional memory efficiency and highly predictable execution latency. The optimized executable allocated 538,621,976 bytes in the heap with a minimal maximum residency of 44,328 bytes and a total active memory footprint of 627 MiB. Garbage collection (GC) overhead was negligible, requiring only a single Generation 1 collection. The maximum recorded GC pause was bounded at 0.0010 seconds and occurred exclusively during program termination (can be seen in the tot time section), confirming that active safety-evaluation loops are completely insulated from garbage collection pauses.

The benchmark completed 1,000,000 continuous safety-evaluation cycles in 0.282 seconds of

mutator (MUT) time, yielding an average decision latency of approximately 282 nanoseconds per cycle. Across repeated testing trials, latency fluctuated tightly between 240 nanoseconds and 350 nanoseconds. This confirms that the compiled modal-depth enforcement loop operates reliably within a predictable, deterministic, sub-microsecond regime. Figure 6 places this latency against the 1 μs sub-microsecond ceiling.

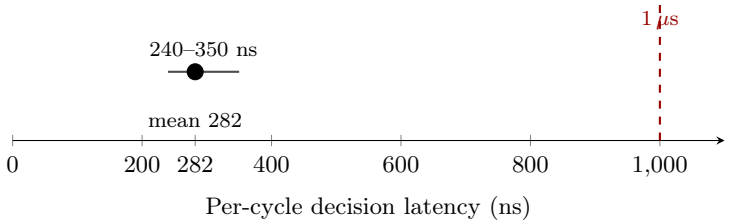


Figure 6: Per-cycle decision latency. The mean (282 ns) and observed range (240–350 ns) remain well below the 1 μs sub-microsecond range.

The benchmark was run on an Apple MacBook Air (M1, 8 GB RAM; 4 performance cores, 4 efficiency cores) using GHC 3.7.1. On real ADS hardware with dedicated computer architectures etc., it is expected that this method will get faster and more efficient times than what was observed on the Macbook Air.

3.5 Summary of Experimental Results

In summary, the empirical evaluations demonstrate that the proposed framework:

- Achieves 100% decision accuracy relative to observable runtime states, establishing its reliability with available real-time data.
- Experiences a 5% accuracy bottleneck under external validation entirely due to environmental epistemic uncertainty.
- Successfully isolates the root cause of boundary execution mismatches to aleatoric uncertainty in the form of localized logical jitter.
- Maintains a predictable sub-microsecond decision window averaging 282 nanoseconds per evaluation cycle.

4 Discussion

4.1 Core Finding and Significance of the Architecture

The central finding of this study is that the proposed modal-depth architecture achieved approximately 100% enforcement accuracy when evaluated relative to the information available to the system at runtime while maintaining sub-microsecond decision latencies. This result suggests that the architecture successfully performed its intended function: enforcing safety constraints through modal evaluation before action execution. Under the observed-context evaluator, no decision-logic failures were observed, indicating that the enforcement mechanism consistently produced the action that was correct relative to the system's internal representation of the environment.

Equally significant is the distinction between architectural correctness and informational correctness. The architecture demonstrated that safety enforcement can be implemented as an endogenous property of the decision-making process rather than as an external monitoring layer. In this

framework, modal enforcement is not appended after a decision has been produced; instead, it is embedded directly into the reasoning process itself. As a result, safety constraints become part of the architecture's operational logic rather than a secondary validation stage.

The observed performance therefore supports the broader hypothesis that modal depth can function as a practical safety axis for autonomous decision systems. While this study does not claim that modal enforcement completely solves autonomous safety, the results indicate that the architecture can reliably enforce decision constraints within the information available to the system while preserving computational efficiency suitable for real-time applications.

4.2 Theoretical Interpretation of the 5% Bottleneck

A key result of the study was the persistent discrepancy between approximately 100% accuracy under the observed-context evaluator and approximately 95% accuracy under the ground-truth evaluator. Throughout development, numerous potential causes were investigated, including threshold selection, stochastic noise handling, modality formulation, runtime optimization, and policy design. Despite repeated modifications to these components, the discrepancy remained largely unchanged.

This persistence suggests that the remaining error is not primarily caused by deficiencies in the modal architecture itself. Instead, the evidence indicates that the bottleneck originates from epistemic uncertainty [4]. The architecture can only reason over the information available to it. When the system's internal representation of context differs from the underlying ground-truth context, disagreement between evaluator and architecture becomes unavoidable even if the architecture behaves perfectly relative to its own knowledge state.

From a theoretical perspective, this distinction is important because it separates architectural reliability from environmental observability. The modal-depth framework appears capable of eliminating internal enforcement failures, but it cannot eliminate uncertainty arising from imperfect knowledge of the environment. Thus, the remaining performance gap should be interpreted as a manifestation of epistemic uncertainty rather than evidence of architectural weakness or aleatoric uncertainty.

The logical jitter existent near decision boundaries is therefore an artifact of aleatoric uncertainty rather than epistemic uncertainty. Small deviations between observed context and ground-truth context occasionally caused the evaluator and the runtime system to disagree about whether a scenario should be classified as dangerous. Noise variations cause `observed_context` values to oscillate across decision boundaries. The logical jitter therefore reflects normal noise variation rather than an information-gap problem.

Just because the `observed_context` value is generated using noise added onto the ground truth value does not mean that the 5% gap is caused by aleatoric uncertainty. Aleatoric Uncertainty represents the noise present in the system that creates the system's interpretation of the ground truth. The 5% bottleneck is still an artifact of epistemic uncertainty because the knowledge gap that arises between the system's interpretation of the ground truth (`observed_context`) and the ground truth (caused by aleatoric uncertainty) is what caused the accuracy loss, rather than the aleatoric uncertainty alone. In other words, aleatoric uncertainty only causes epistemic uncertainty and noise variations in the system. This should not be used as a formal or universal definition of epistemic and aleatoric uncertainty, it is just the nature of how they work individually and together in this sandbox safety method because of how intertwined aleatoric and epistemic uncertainty are in this safety method.

4.3 Alternative Strategies and Architectural Tradeoffs

Observed-Context Evaluation. The final implementation adopted an evaluator based on observed context rather than ground-truth context. This choice was motivated by the objective of determining whether the architecture was performing correctly relative to the information available during execution.

Evaluating against observed context measures the architecture on the basis of what it actually knows rather than information that exists only from the perspective of an external observer. Because autonomous systems must make decisions using incomplete information, this evaluation methodology provides a direct assessment of architectural correctness under realistic operating conditions. The resulting approximately 100% enforcement accuracy therefore demonstrates that the modal-depth framework can successfully execute its intended decision process when supplied with the information available to it.

Additionally, this evaluation strategy provides evidence that the architecture can be implemented in practical systems. Real autonomous systems do not have access to perfect knowledge of the world; they operate using internal representations. An evaluator based on observed context therefore measures the architecture under conditions that represent the deployment environment instead of the real world environment.

Hysteresis-Based Evaluation. An alternative approach identified during experimentation was the introduction of hysteresis into the evaluator [10]. Under this strategy, entering a danger state and exiting a danger state require crossing different regimes or thresholds, thereby creating a stable decision band around boundary regions.

Hysteresis directly addresses logical jitter by reducing oscillatory state transitions caused by small fluctuations near threshold values. Because epistemic uncertainty often produces marginal disagreements between perceived and actual context, a hysteresis-based evaluator may provide a more stable representation of system behavior under uncertain conditions.

Unlike the observed-context evaluator, hysteresis preserves evaluation against ground-truth context while introducing tolerance for aleatoric uncertainty near decision boundaries.

False-Positive Safety Strategy. A further architectural strategy involves intentionally biasing residual uncertainty toward false positives rather than false negatives. Under such a design, uncertain situations would preferentially result in action denial rather than action approval.

Although false positives remain undesirable because they restrict actions that may in fact be safe, their consequences are generally less severe than false negatives in safety-critical environments [2], [5]. A system that occasionally refuses safe actions may experience reduced operational efficiency, but a system that occasionally permits dangerous actions can experience catastrophic failure.

Therefore, future implementations may benefit from incorporating conservative fallback policies that ensure uncertainty is resolved in favor of safety. Such mechanisms would not necessarily improve measured accuracy but could significantly improve practical robustness under ground-truth evaluation. This tradeoff highlights an important design consideration: maximizing operational permissiveness is not always equivalent to maximizing system safety.

4.4 Computational Implications

Beyond correctness, the experimental results demonstrated exceptionally low computational overhead. Performance testing measured approximately 282 nanoseconds per decision cycle, placing the architecture well within the sub-microsecond range.

This result has several implications. First, it indicates that modal-depth enforcement can be integrated into decision pipelines without introducing substantial computational burden. Safety

mechanisms are often criticized for increasing latency or reducing system responsiveness. The measured execution time suggests that the proposed architecture avoids this limitation.

Second, runtime profiling revealed no observable garbage-collection interruptions during the enforcement process. This insulation from garbage-collection pauses is significant because unpredictable execution delays can be problematic in systems that require deterministic behavior. The absence of measurable interruption indicates that the architecture maintains consistent runtime characteristics under the tested workload.

Third, the measured latency places the architecture within the performance range required by high-frequency decision systems. At approximately 282 nanoseconds per evaluation cycle, the enforcement mechanism is capable of operating at rates substantially faster than many practical decision loops. The computational cost of modal enforcement is therefore unlikely to be the limiting factor in deployment scenarios.

Taken together, these findings suggest that modal-depth safety enforcement is not merely theoretically feasible but also computationally practical for systems requiring rapid decision execution.

4.5 Study Limitations

Several limitations should be considered when interpreting the results of this study.

The first limitation concerns scenario generation. Experimental scenarios were generated using Haskell's `randomRIO` uniform distributions. While this approach provides broad coverage across the state space and facilitates controlled experimentation, real autonomous environments may not exhibit uniformly distributed conditions. Operational systems often encounter skewed, clustered, heavy-tailed, Gaussian, non-Gaussian, or otherwise non-uniform distributions [3], [4]. Similarly, performance observed under uniform sampling may not fully represent behavior under alternative statistical environments.

A related limitation is the treatment of uncertainty. Although epistemic uncertainty was explicitly modeled within the experimental framework, the implementation was tested within a sandbox environment rather than a deployed autonomous decision system. Real systems may involve more complex uncertainty effects than observed in this sandbox model. As a result, the magnitude and behavior of epistemic uncertainty and aleatoric uncertainty observed in practice may differ from those measured in this study.

Another limitation concerns the interpretation of observed-context accuracy. While the architecture achieved approximately 100% enforcement accuracy relative to runtime information, this result does not imply perfect alignment with environmental reality. The remaining discrepancy under ground-truth evaluation demonstrates that these information limitations continue to influence system performance by creating a mismatch between the system's observed context and the real world ground truth. Even with careful tuning, some degree of divergence between perception and reality may persist.

Most importantly, the architecture has not yet been validated within a real autonomous decision system [3]. The current study demonstrates a practical method within a controlled simulation environment, but deployment introduces operational complexities that cannot be fully captured through sandbox experimentation alone. Therefore, conclusions regarding real-world performance should be considered provisional until validated through implementation in practical autonomous systems.

4.6 Future Work

Several directions for future research emerge from these findings.

The most immediate objective is to address the epistemic uncertainty problem observed in this safety method. Although the safety method has already proved to be feasible, this does not mean it can be deployed in an ADS immediately without further testing and refinement. It only indicates that the idea could work; it still requires additional work to reach its intended purpose. In the sandbox, epistemic uncertainty arises from the gap in knowledge between the `observed_context` and the ground-truth context. Because a real implementation would not define modalities in terms of `observed_context` and would instead use true action-pipeline signals as modalities, epistemic uncertainty may manifest differently or be reduced in a real ADS. This should be investigated carefully before designing solutions for the epistemic-uncertainty problem: the fact that it exists in the sandbox due to the way the safety method is defined there does not guarantee that it will appear in the same way, or at all in a real implementation. It is based on the evaluation method used to evaluate the safety method now that ground truth wont exist in the same way or at all.

Future studies should also investigate alternative statistical environments. In particular, experiments incorporating Gaussian, non-Gaussian, heavy-tailed, and domain-specific distributions would help determine the extent to which modal-depth performance depends on the structure of the underlying environment. Understanding how the architecture behaves under various uncertainty models will be essential for characterizing the safety method's capabilities.

A further objective is deployment within a real Autonomous Decision System. Such experimentation would provide direct evidence regarding the architecture's behavior under operational conditions and would allow evaluation of the framework against real decision-making workloads.

Human deep neural environments and processes have been shaped by millions of years of evolution and adaptation. By drawing direct inspiration from these biological mechanisms for safety enforcement, behavioral regulation, and response to perceived danger, future work may improve the structure of the proposed safety method. These bio-inspired principles could support the development of a more efficient and effective framework enforcing safety in a effective and more efficient way.

So, future work during deployment in real autonomous decision systems should also take inspiration from deep neural processes that occur in the human brain. These processes already share high-level similarities with the safety method developed here: both use multiple interacting signals (modalities), map the state of the system (body and brain versus ADS and action pipeline), apply tunable thresholds, and perform high/low danger evaluation (deciding when an action is dangerous and when it is not). This research direction should aim to make the safety method increasingly aligned with these neural safety processes for improved effectiveness and efficiency in safety enforcement. An initial focus should be on refining the policy and the thresholds, as these are the most important components of the safety method after the modalities. The current modality design may be sufficient, but it can be modified if deployment reveals limitations, or if it is needed for the thresholds to understand and be able to give proper danger level data. Taking direct inspiration from these deep neural processes and implementing it into the safety method can pave the way for more advanced studies regarding limiting AI system's abilities, just like how the human brain is able to limit human abilities.

Additional research should examine hysteresis and related uncertainty-management techniques as potential mechanisms for reducing epistemic uncertainty. Comparative studies between observed-context evaluation, hysteresis-based evaluation, and hybrid approaches may reveal strategies that further improve reliability under uncertain conditions.

Hysteresis should be implemented as a threshold strategy, if not an evaluation method. Instead of using gates in the evaluation method, using it in the threshold takes inspiration from human deep neural environment safety processes, and can thus increase efficiency and possibly remove the need for a tri-threshold approach because of the way it is structured (using gates as a way to determine

high and low danger levels instead of using one threshold value where logical jitter and effects of aleatoric uncertainty can occur).

The framework presented here is intended as a foundation rather than a finished system. Testing with real Autonomous Decision Systems will require subsequent modifications to the safety pipeline so that the modal structure, thresholds, and enforcement behavior match the requirements of the specific ADS into which it is deployed.

4.7 Implementation as an Endogenous Safety Enforcement Method

Implementation as an Endogenous Safety Enforcement Architecture

The results of this study support implementation of modal depth as an endogenous safety enforcement architecture. Under this model, safety is not treated as an external monitoring function that evaluates actions after they have been generated. Instead, safety constraints are embedded directly within the decision-making architecture itself or coupled with the decision-making architecture. This safety method will exist in the software logic and cognitive reasoning portion of the ADS, which deals with actions, logic, and reasoning. Integration into this portion of the ADS will allow the safety method to operate within or be coupled to the action pipeline.

In practice, each decision is evaluated through the active modal structure before action execution. The specific modalities used are not hardcoded by the experiment, but are derived from the requirements, inputs, and operational logic of the autonomous decision system being studied. In a real autonomous decision system, modalities are defined by treating specific components of the action pipeline as modalities, and repeating this for each modality needed to enforce safety accurately. The reasoning module, ethical module, contextual module, and safety module are examples of system-dependent modal dimensions that are defined differently based on the ADS, collectively determining whether an action satisfies the required safety conditions; modality definitions are based on the system.

The safety enforcement process should occur as part of the system's action pipeline, rather than as an independent supervisory mechanism. This design offers several advantages. Because enforcement is integrated into the architecture, unsafe actions can be prevented before execution rather than detected after the fact. Additionally, the low computational overhead achieved by the safety method in testing suggests that endogenous safety enforcement can be maintained without sacrificing safety method responsiveness. Integrating enforcement into the decision-making architecture or coupling it with that architecture should theoretically make the safety method operate more efficiently and effectively compared to integration in a different part of the ADS or outside of the ADS.

More broadly, the architecture re-frames safety as a structural property of decision-making rather than an externally imposed constraint. The results indicate that modal-depth enforcement can function as an internal governance mechanism that could be capable of maintaining high levels of enforcement accuracy while preserving computational performance across different autonomous decision systems, provided that the modal structure is instantiated in accordance with the system's own operational requirements. Although further validation in real autonomous decision systems remains necessary, findings prove that endogenous safety architectures are a viable direction for safety enforcement in autonomous decision systems.

Also, during implementation, the safety method should be further refined by taking inspiration from deep neural processes in the brain as shown in the future work subsection.

4.7.1 How to Implement in Real ADS Systems

To implement modal depth in a real autonomous decision system, the architecture should be inserted directly into the action pipeline rather than added as an external safety layer.

A practical implementation begins with defining the system's modal inputs. These inputs should include whatever is considered as a danger evaluator for the system. As covered in subsection 2.3, modalities should be defined as just defining a part of the action pipeline as a modality, and doing this for every other modality needed in order to enforce safety accurately.

The modal evaluator then computes whether the action is permissible based on what is defined as correct for the system. Based on system requirements, if the high danger score and low danger score are defined, then the system should act accordingly. Also, if the system is under a point of uncertainty, there should be a fallback included to make sure the effects of uncertainty affect the system less.

In a deployed autonomous decision system, this enforcement layer should operate synchronously or faster than the action-selection process. For example, in a robotics or vehicle-control setting, the planner may generate a candidate maneuver, but the modal evaluator must verify that the maneuver remains safe given the system's current knowledge. If uncertainty is high or the environment is only partially observed, then a possible solution as discussed would be for the architecture to bias toward safety by rejecting ambiguous actions or selecting lower-risk alternatives. This ensures that uncertainty is handled at the point of decision rather than after the fact. If it were handled after the fact, the system would be ineffective in enforcing safety.

Finally, real-world deployment should test the system created in the sandbox in a real autonomous decision system in sample tests. The modal-depth layer should be validated under nominal conditions, boundary conditions, and adversarial or noisy inputs to ensure that it behaves consistently across a wide range of scenarios. Once integrated, the architecture functions as an endogenous safety mechanism: it does not merely observe the system's behavior, but actively shapes which actions are permitted to occur.

The architecture itself remains unchanged across deployments, but what `ACT`ing to enforce safety does and how it is performed must be manually added based on the system in question; this is not included in the sandbox.

Choosing and tuning thresholds

Thresholds in the safety method (e.g., modality-specific danger thresholds, confidence weights, and enforcement triggers) should be selected in two stages:

1. **Empirical tuning in the sandbox.** Once an analytically motivated range is established, use the sandbox to refine thresholds. This can be done in one of two ways:

   - **Automated tuning via `tunethresholds.hs`.** The sandbox includes the script `tunethresholds.hs`, which systematically searches over threshold values within specified ranges and evaluates performance metrics (e.g., accuracy, false-positive/false-negative rates, stability). This provides an automated way to identify threshold configurations that maximize desired objectives under the sandbox's noise and uncertainty models.
   - **Manual hyperparameter tuning.** Alternatively, thresholds can be tuned by selecting a starting value and iteratively adjusting them across runs. In each run, the threshold values are modified based on observed performance. This will be continued until the configuration that yields the highest accuracy (or other chosen metric) is determined.

If the gap between the real world and the system's perception of reality is approximately fixed and well-characterized, threshold values can be relatively coarse and robust, as they do not need to track fine-grained variations in that gap. If, however, the perception–reality gap varies significantly across scenarios or over time, thresholds may need to be more carefully tuned or made adaptive.

*This is the way thresholds are tuned in the sandbox safety method. In a real implementation, this threshold tuning method **will not work** if there is no evaluator or evaluation method to evaluate whether the safety method works or not. If there is no evaluation method then further research will need to be conducted in order to determine the proper way to determine threshold values in that ADS, and a proper way to make an evaluation method based on the ADS the safety method is being implemented in.

4.8 Sandbox Testing

The sandbox environment is designed to support the **design and tuning of the safety method**, not to serve as a test harness for any particular Autonomous Decision System. Its purpose is to provide a controlled, customizable setting in which the safety method can be exercised under known levels of uncertainty and noise, so that threshold values, enforcement rules, and other safety method parameters can be refined before integration into a specific ADS. The sandbox is therefore not intended to replace ADS-specific testing; it is intended to enable principled, low-risk tuning of the threshold and the testing of safety method's parameters prior to deployment.

The sandbox should be used **only** for the following purposes:

- **Testing new additions and modifications to the safety method's architecture or logic.** When new mechanisms (e.g., additional modalities, modified enforcement rules, alternative thresholding strategies, hysteresis policies, or other architectural changes) are proposed, the sandbox allows them to be evaluated under controlled conditions, independent of any specific ADS. This should be the place where new architectural additions or modifications are tested and are made sure to work.

The sandbox should have **no role in testing the Autonomous Decision System**. System-level validation—i.e., testing whether the ADS as a whole behaves safely and effectively under real operational conditions—must be performed in the **ADS-specific testing environment**.

When testing the implementation in the ADS specific testing environment, if the autonomous decision system tested is unpredictable and difficult to contain, take precautions and first compare this implementation to the older implementation to get an idea of the parameters of how the possible outcomes of the testing to see if testing is safe or not, and if extra precautions or additions onto the safety method are needed for that specific ADS being tested.

Testing whether the ADS as a whole behaves safely and effectively under real operational conditions must be performed in the **ADS-specific testing environment**.

Code Availability

The Haskell sandbox implementation is publicly available at:

https://github.com/anish-chilakala/modal-depth-ADS-safety-method

The repository is licensed under the MIT License. The accompanying paper and documentation are licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).

References

[1] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1*, 2023. https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf

[2] Amodei, D., Olah, C., Steinhardt, J., et al. *Concrete Problems in AI Safety.* arXiv:1606.06565, 2016. https://arxiv.org/abs/1606.06565

[3] Hendrycks, D., Carlini, N., Schulman, J., and Steinhardt, J. *Unsolved Problems in ML Safety.* arXiv:2109.13916, 2021. https://arxiv.org/abs/2109.13916

[4] Kendall, A. and Gal, Y. *What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?* arXiv:1703.04977, 2017. https://arxiv.org/abs/1703.04977

[5] Alshiekh, M., Bloem, R., Ehlers, R., Könighofer, B., Niekum, S., and Topcu, U. *Safe Reinforcement Learning via Shielding.* In Proc. AAAI Conference on Artificial Intelligence, 2018. https://ojs.aaai.org/index.php/AAAI/article/view/11797

[6] Könighofer, B., Bloem, R., Jansen, N., Junges, S., and Pranger, S. *Shields for Safe Reinforcement Learning.* ACM Computing Surveys, 2025. https://dl.acm.org/doi/10.1145/3715958

[7] Sha, L. *Using Simplicity to Control Complexity.* IEEE Software, 18(4):20–28, 2001. https://www.computer.org/csdl/magazine/so/2001/04/s4020/13rRUzphDvS

[8] Anthropic. *Transparency Hub: Model Report* (documents Claude Mythos Preview, including its ability to autonomously discover and exploit zero-day vulnerabilities, and its restricted release). https://www.anthropic.com/transparency/model-report

[9] Anthropic. *Alignment Risk Update: Claude Mythos Preview.* April 7, 2026. https://www-cdn.anthropic.com/79c2d46d997783b9d2fb3241de43218158e5f25c.pdf

[10] Åström, K. J. and Murray, R. M. *Feedback Systems: An Introduction for Scientists and Engineers*, 2nd ed. Princeton University Press, 2021. Available at https://fbsbook.org